

Wheels Alpes

Diseño de la base de datos — Entrega 1

ISIS1511 — Fundamentos de Bases de Datos

Universidad de los Andes

13 de septiembre de 2026

Jose Luis Rodriguez Leon

Roxana Soto Noriega

Sara Ortiz Ortiz

Grupo 7

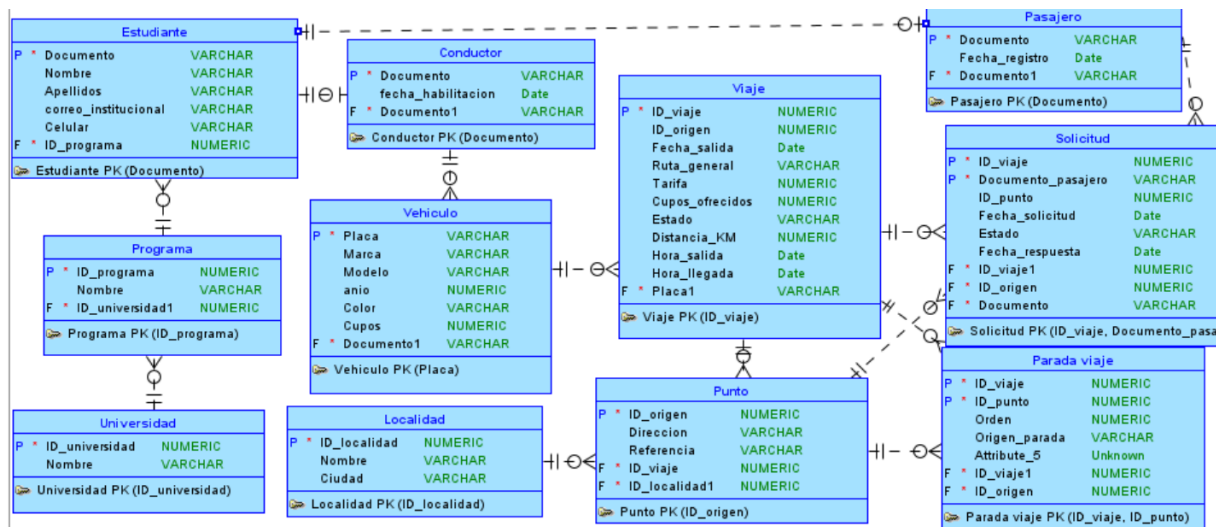
1. De qué se trata

Wheels Alpes es una plataforma de carpooling para estudiantes universitarios. Un estudiante con carro publica un viaje hacia o desde la universidad, dice a qué hora sale, por dónde pasa, cuánto cobra y cuántos cupos ofrece. Otros estudiantes buscan viajes, piden un cupo y eligen dónde quieren que los recojan. El conductor decide si acepta o rechaza cada solicitud.

A partir de eso diseñamos la base de datos. Este documento tiene tres partes: el modelo conceptual (qué entidades existen y cómo se relacionan), el modelo relacional (cómo se convierte en tablas y por qué queda normalizado) y las pruebas con las que verificamos que el diseño funciona.

2. Modelo conceptual (E/R)

2.1 Diagrama



2.2 Entidades

Nos quedaron once entidades. Estas son y por qué las incluimos:

Entidad	Llave	Por qué existe
UNIVERSIDAD	id_universidad	RF1 y RF7 piden guardar la universidad del estudiante. Como entidad aparte evitamos repetir el nombre en cada estudiante.
PROGRAMA	id_programa	También lo piden RF1 y RF7. Cada programa pertenece a una universidad, así que sabiendo el programa ya sabemos la universidad.
ESTUDIANTE	documento	Los datos que pide el enunciado para conductores y para pasajeros son exactamente los mismos, así que los juntamos aquí. El documento sirve de llave porque es único y no cambia.
CONDUCTOR	documento	Es el único que puede inscribir vehículos (RF2) y publicar viajes (RF3).
PASAJERO	documento	Es el único que puede pedir cupos (RF10) y cancelarlos (RF11).
VEHICULO	placa	RF2. Usamos la placa como llave porque es única y no cambia.
VIAJE	id_viaje	Es el centro del modelo. RF3 lo crea, RF5 le cambia el estado, RF6 y RF9 lo consultan.

Entidad	Llave	Por qué existe
PUNTO	id_punto	El enunciado habla de tres cosas que en realidad son lo mismo: el punto de origen del conductor, las paradas y el punto de recogida del pasajero. Los unificamos en una sola entidad.
LOCALIDAD	id_localidad	Sin esto no podríamos resolver RF9 (buscar por zona), RFC1, RFC2, RFC3 ni RFA2, que agrupan por localidad.
PARADA_VIAJE	id_viaje + id_punto	Un viaje tiene varias paradas y un punto puede ser parada de varios viajes. Es M:N, así que necesita su propia tabla.
SOLICITUD	id_viaje + documento_pasajero	Un pasajero pide cupo en varios viajes y un viaje recibe solicitudes de varios pasajeros. También es M:N.

2.3 Atributos

Entidad	Atributos
UNIVERSIDAD	id_universidad, nombre
PROGRAMA	id_programa, nombre, id_universidad
ESTUDIANTE	documento, nombre, apellidos, correo_institucional, celular, id_programa
CONDUCTOR	documento, fecha_habilitacion
PASAJERO	documento, fecha_registro
VEHICULO	placa, marca, modelo, anio, color, cupos, documento_conductor
VIAJE	id_viaje, fecha_hora_salida, ruta_general, tarifa, cupos_ofrecidos, estado, distancia_km, hora_salida_real, hora_llegada_real, placa, id_punto_origen
PUNTO	id_punto, direccion, referencia, id_localidad
LOCALIDAD	id_localidad, nombre, ciudad
PARADA_VIAJE	id_viaje, id_punto, orden, origen_parada
SOLICITUD	id_viaje, documento_pasajero, id_punto, fecha_solicitud, estado, fecha_respuesta

Tres atributos que agregamos nosotros. distancia_km, hora_salida_real y hora_llegada_real no salen en el texto del caso, pero RFA1 pide el tiempo total y la distancia del conductor, y RFA2 pide el promedio de distancia y tiempo por viaje. Sin guardarlos no hay forma de calcular los tableros.

Un atributo que decidimos NO guardar. Los cupos que quedan libres en un viaje no se almacenan: se calculan restando las solicitudes aceptadas a cupos_ofrecidos. Es un dato derivado (DD) y guardarlo sería repetir información que se puede sacar sola.

2.4 Relaciones

Relación	Cardinalidad	Obligatorio	Por qué
UNIVERSIDAD ofrece PROGRAMA	1:N	Sí del lado programa	Todo programa es de alguna universidad, pero una universidad puede estar registrada sin programas.
PROGRAMA — ESTUDIANTE	1:N	Sí del lado estudiante	RF1 y RF7 piden el programa, o sea que es obligatorio.
ESTUDIANTE es un CONDUCTOR	1:1	No	No todo estudiante es conductor.

Relación	Cardinalidad	Obligatorio	Por qué
ESTUDIANTE es un PASAJERO	1:1	No	No todo estudiante es pasajero.
CONDUCTOR inscribe VEHICULO	1:N	Sí en los dos lados	Todo carro tiene dueño, y el enunciado dice que uno solo se registra como conductor si puede inscribir un vehículo.
VEHICULO — VIAJE	1:N	Sí del lado viaje	RF3 dice que hay que asociar un vehículo al viaje. Un carro puede no tener viajes todavía.
PUNTO es origen de VIAJE	1:N	Sí del lado viaje	Todo viaje arranca en algún lado.
LOCALIDAD agrupa PUNTO	1:N	Sí del lado punto	Si un punto no tiene localidad, RF9 y los RFC no funcionan.
VIAJE tiene PARADA_VIAJE	1:N	Sí en los dos lados	RF3 pide paradas iniciales, así que un viaje sin paradas no debería existir.
PUNTO — PARADA_VIAJE	1:N	Sí del lado parada	Una parada siempre apunta a un punto que ya existe.
VIAJE recibe SOLICITUD	1:N	Sí del lado solicitud	Un viaje puede quedarse sin solicitudes.
PASAJERO hace SOLICITUD	1:N	Sí del lado solicitud	Hay que estar autenticado para pedir cupo.
PUNTO — SOLICITUD	1:N	Sí del lado solicitud	RF10 dice que al pedir cupo hay que elegir el punto de recogida.

2.5 La herencia

CONDUCTOR y PASAJERO son subtipos de ESTUDIANTE. La especialización es total y con traslape.

- Total: cualquiera que se registre en la plataforma lo hace para conducir o para viajar, no hay estudiantes "sueltos".
- Con traslape: el mismo estudiante puede ser conductor en los viajes que publica y pasajero en los de otros. El enunciado no lo prohíbe y es lo normal en una app de carpooling.

Esto importa porque el traslape decide cómo se convierte la herencia en tablas más adelante (sección 3.3).

2.6 Reglas de negocio

Hay condiciones del negocio que el diagrama E/R no puede dibujar. Las anotamos aparte para no perderlas:

#	Regla	Dónde se controla
RN1	Los cupos que ofrece un viaje no pueden ser más que los cupos del carro.	Trigger
RN2	No se pueden aceptar más solicitudes que cupos ofrecidos.	Trigger
RN3	Solo se puede pedir cupo en viajes que están PROGRAMADO.	Trigger
RN4	Para cancelar un viaje o una solicitud hay que hacerlo con dos horas de anticipación.	Trigger
RN5	Los estados van en orden: PROGRAMADO, EN_CURSO, COMPLETADO. Se puede cancelar desde los dos primeros. COMPLETADO y CANCELADO ya no se cambian.	Trigger
RN6	Si se cancela un viaje, sus solicitudes aceptadas quedan canceladas también.	Trigger

#	Regla	Dónde se controla
RN7	El conductor no puede pedir cupo en su propio viaje.	Trigger
RN8	Cuando el conductor acepta una solicitud, el punto de recogida se vuelve parada del viaje.	Trigger
RN9	Un pasajero no puede tener dos solicitudes activas para el mismo viaje.	La PK compuesta de SOLICITUD ya lo impide
RN10	Un conductor tiene que tener al menos un carro inscrito.	Aplicación
RN11	El documento y el correo institucional no se pueden repetir.	PK y restricción ND

Un problema con RN10. Si decimos que un conductor necesita un carro y que un carro necesita un conductor, no podemos insertar ninguno de los dos primero. Es el mismo enredo que pasa cuando una relación 1:1 es obligatoria por los dos lados. Por eso esta regla no se puede poner como restricción de la base de datos y toca validarla en la aplicación, dentro de la misma transacción de registro.

2.7 Supuestos

- Un estudiante está en un solo programa a la vez.
- Cada carro tiene un solo dueño.
- Un conductor puede tener varios carros, pero cada viaje usa uno solo.
- La tarifa es la misma para todos los pasajeros del viaje, como dice el enunciado.
- La distancia y las horas reales solo se llenan cuando el viaje termina, así que antes quedan en nulo.
- Un mismo punto lo pueden usar varios viajes y varios pasajeros.
- No modelamos pagos. Los ingresos se calculan sumando las tarifas de las solicitudes aceptadas de viajes completados.

2.8 Qué entidades usa cada requerimiento

Requerimiento	Entidades
RF1 Registrar conductor	ESTUDIANTE, CONDUCTOR, PROGRAMA, UNIVERSIDAD
RF2 Registrar vehículo	VEHICULO, CONDUCTOR
RF3 Publicar viaje	VIAJE, VEHICULO, PUNTO, PARADA_VIAJE
RF4 Gestionar solicitudes	SOLICITUD, PARADA_VIAJE, VIAJE
RF5 Actualizar estado	VIAJE, SOLICITUD
RF6 Historial de viajes	VIAJE, SOLICITUD, PASAJERO
RF7 Crear cuenta pasajero	ESTUDIANTE, PASAJERO, PROGRAMA
RF8 Gestionar cuenta	ESTUDIANTE
RF9 Ver viajes disponibles	VIAJE, PARADA_VIAJE, PUNTO, LOCALIDAD
RF10 Pedir cupo	SOLICITUD, PUNTO, VIAJE, PASAJERO
RF11 Cancelar solicitud	SOLICITUD, VIAJE
RFC1 Demanda por localidad	SOLICITUD, PUNTO, LOCALIDAD

Requerimiento	Entidades
RFC2 Ingresos por localidad	VIAJE, SOLICITUD, PUNTO, LOCALIDAD
RFC3 Viajes por conductor	VIAJE, VEHICULO, CONDUCTOR, PUNTO, LOCALIDAD
RFA1 Tablero conductor	VIAJE, SOLICITUD, VEHICULO, CONDUCTOR
RFA2 Tablero pasajero	SOLICITUD, VIAJE, PUNTO, LOCALIDAD

3. Del E/R a las tablas (algoritmo de Chen)

Aplicamos el algoritmo visto en clase, paso por paso.

3.1 Entidades que pasan directo a tabla

Siete entidades son independientes y se convierten en tabla sin más, cada una con su llave:

Tabla	Llave primaria	Quién la asigna
UNIVERSIDAD	id_universidad	El sistema (SA)
PROGRAMA	id_programa	El sistema (SA)
ESTUDIANTE	documento	El usuario (UA)
VEHICULO	placa	El usuario (UA)
VIAJE	id_viaje	El sistema (SA)
PUNTO	id_punto	El sistema (SA)
LOCALIDAD	id_localidad	El sistema (SA)

En ESTUDIANTE le pusimos ND al correo institucional porque no se puede repetir (RN11) y eso lo vuelve una llave candidata.

3.2 Atributos multivalor

Los atributos multivalor no se permiten porque rompen la primera forma normal. En nuestro caso el candidato eran las paradas: un viaje tiene varias y meterlas todas en una casilla de VIAJE habría quedado mal. Por eso las sacamos a la tabla PARADA_VIAJE.

No encontramos otros. Asumimos un solo celular por estudiante, porque el enunciado lo menciona en singular. Si hicieran falta varios, la solución sería igual: una tabla aparte con FK hacia ESTUDIANTE.

3.3 Herencia

El algoritmo da tres opciones para convertir una especialización. Miramos las tres:

Opción	En qué consiste	Qué pasó
Alternativa 0	Una sola tabla con todo y una columna que diga qué rol es.	No sirve. Como hay traslape, un estudiante que sea conductor y pasajero necesitaría dos valores en esa columna al tiempo.
Alternativa 1	Una tabla por cada subtipo, repitiendo los datos del estudiante en las dos.	No sirve. Los datos del que tiene los dos roles quedarían repetidos, y al actualizarlos con RF8 podrían quedar distintos en cada tabla.
Alternativa 2	Se mantiene la tabla ESTUDIANTE y cada subtabla lleva el documento como PK y FK al mismo tiempo.	Esta escogimos. Es la que recomiendan para herencia con traslape: el documento puede estar en CONDUCTOR y en PASAJERO a la vez, y los datos personales quedan en un solo lugar.

3.4 Relaciones 1:1

No tenemos relaciones 1:1 entre entidades independientes. Las únicas 1:1 son las de la herencia, que ya se resolvieron en el paso anterior.

3.5 Relaciones 1:N

La llave del lado "uno" se pasa como FK al lado "muchos". Cuando el lado "uno" es obligatorio, esa FK lleva NN.

Relación	FK que se pasa	A qué tabla	Restricción
UNIVERSIDAD — PROGRAMA	id_universidad	PROGRAMA	FK UNIVERSIDAD.id_universidad, NN
PROGRAMA — ESTUDIANTE	id_programa	ESTUDIANTE	FK PROGRAMA.id_programa, NN
CONDUCTOR — VEHICULO	documento_conductor	VEHICULO	FK CONDUCTOR.documento, NN
VEHICULO — VIAJE	placa	VIAJE	FK VEHICULO.placa, NN
PUNTO — VIAJE (origen)	id_punto_origen	VIAJE	FK PUNTO.id_punto, NN
LOCALIDAD — PUNTO	id_localidad	PUNTO	FK LOCALIDAD.id_localidad, NN

Las seis llevan NN porque en todos los casos el lado "uno" es obligatorio.

3.6 Relaciones M:N

Las M:N hay que romperlas con una tabla intermedia cuya llave primaria son las dos FK juntas. Los atributos de la relación se van a esa tabla nueva.

Relación M:N	Tabla nueva	Llave primaria	Atributos propios
VIAJE — PUNTO	PARADA_VIAJE	id_viaje + id_punto	orden, origen_parada
PASAJERO — VIAJE	SOLICITUD	id_viaje + documento_pasajero	fecha_solicitud, estado, fecha_respuesta, más la FK id_punto

SOLICITUD queda con una tercera FK, id_punto, que no hace parte de la llave. Es el punto de recogida que elige el pasajero (RF10) y es un dato propio de esa solicitud.

Algo que decidimos aquí: solicitud y reserva son la misma tabla. Una reserva no es más que una solicitud que el conductor ya aceptó, así que lo manejamos con la columna estado en vez de crear dos tablas.

3.7 Relaciones recursivas

No hay. Ninguna entidad se relaciona consigo misma, así que este paso no aplica.

3.8 Modelo relacional

Convenciones: PK llave primaria, FK llave foránea (se indica a qué apunta), NN no acepta nulos, ND no acepta duplicados, CK restricción de chequeo, SA lo asigna el sistema, UA lo asigna el usuario, DD dato derivado.

Tabla	Atributo	Restricciones
UNIVERSIDAD	id_universidad	PK, NN, ND, SA
	nombre	NN, ND
PROGRAMA	id_programa	PK, NN, ND, SA
	id_universidad	FK UNIVERSIDAD.id_universidad, NN
	nombre	NN, ND junto con id_universidad
ESTUDIANTE	documento	PK, NN, ND, UA
	id_programa	FK PROGRAMA.id_programa, NN
	nombre	NN
	apellidos	NN
	correo_institucional	NN, ND, UA
	celular	NN
CONDUCTOR	documento	PK, FK ESTUDIANTE.documento, NN, ND
	fecha_habilitacion	NN, SA
PASAJERO	documento	PK, FK ESTUDIANTE.documento, NN, ND
	fecha_registro	NN, SA
VEHICULO	placa	PK, NN, ND, UA
	documento_conductor	FK CONDUCTOR.documento, NN
	marca	NN
	modelo	NN
	anio	NN, CK[anio entre 1950 y el año actual + 1]
	color	NN
	cupos	NN, CK[cupos entre 1 y 20]
VIAJE	id_viaje	PK, NN, ND, SA
	placa	FK VEHICULO.placa, NN
	id_punto_origen	FK PUNTO.id_punto, NN
	fecha_hora_salida	NN
	ruta_general	NN
	tarifa	NN, CK[tarifa >= 0]
	cupos_ofrecidos	NN, CK[cupos_ofrecidos > 0]
	estado	NN, CK[estado en ('PROGRAMADO','EN_CURSO','COMPLETADO','CANCELADO')]
	distancia_km	Acepta nulos hasta que el viaje termine, CK[distancia_km > 0]
	hora_salida_real	Acepta nulos hasta que el viaje arranque

Tabla	Atributo	Restricciones
	hora_llegada_real	Acepta nulos, CK[hora_llegada_real > hora_salida_real]
	cupos_disponibles	DD, no se guarda: cupos_ofrecidos menos solicitudes aceptadas
PUNTO	id_punto	PK, NN, ND, SA
	id_localidad	FK LOCALIDAD.id_localidad, NN
	direccion	NN
	referencia	Acepta nulos: el enunciado dice dirección o punto de referencia
LOCALIDAD	id_localidad	PK, NN, ND, SA
	nombre	NN, ND junto con ciudad
	ciudad	NN
PARADA_VIAJE	id_viaje	PK, FK VIAJE.id_viaje, NN
	id_punto	PK, FK PUNTO.id_punto, NN
	orden	NN, CK[orden > 0], ND junto con id_viaje
	origen_parada	NN, CK[origen_parada en ('INICIAL','SOLICITUD')]
SOLICITUD	id_viaje	PK, FK VIAJE.id_viaje, NN
	documento_pasajero	PK, FK PASAJERO.documento, NN
	id_punto	FK PUNTO.id_punto, NN
	fecha_solicitud	NN, SA
	estado	NN, CK[estado en ('PENDIENTE','ACEPTADA','RECHAZADA','CANCELADA')]
	fecha_respuesta	Acepta nulos mientras esté PENDIENTE

4. Normalización

4.1 Dependencias funcionales y llaves candidatas

Antes de revisar las formas normales hay que decir qué determina qué en cada tabla. Eso es lo que vamos a usar después.

Tabla	Qué determina qué	Llaves candidatas
UNIVERSIDAD	id_universidad → nombre. También nombre → id_universidad	id_universidad, nombre
PROGRAMA	id_programa → nombre, id_universidad. También (nombre, id_universidad) → id_programa	id_programa, (nombre, id_universidad)
ESTUDIANTE	documento → todos los demás. También correo_institucional → todos los demás	documento, correo_institucional
CONDUCTOR	documento → fecha_habilitacion	documento
PASAJERO	documento → fecha_registro	documento
VEHICULO	placa → documento_conductor, marca, modelo, anio, color, cupos	placa
VIAJE	id_viaje → todos los demás	id_viaje
PUNTO	id_punto → id_localidad, direccion, referencia	id_punto
LOCALIDAD	id_localidad → nombre, ciudad. También (nombre, ciudad) → id_localidad	id_localidad, (nombre, ciudad)
PARADA_VIAJE	(id_viaje, id_punto) → orden, origen_parada. También (id_viaje, orden) → id_punto	(id_viaje, id_punto), (id_viaje, orden)
SOLICITUD	(id_viaje, documento_pasajero) → id_punto, fecha_solicitud, estado, fecha_respuesta	(id_viaje, documento_pasajero)

4.2 Primera forma normal

Para estar en 1FN, ninguna casilla puede tener listas ni datos compuestos adentro. Revisamos tabla por tabla:

Tabla	Revisión
UNIVERSIDAD	Bien, los dos atributos son simples.
PROGRAMA	Bien.
ESTUDIANTE	Bien. Separamos nombre y apellidos en vez de poner un solo campo con todo. Un celular por persona.
CONDUCTOR	Bien, una sola fecha.
PASAJERO	Bien, una sola fecha.
VEHICULO	Bien. marca y modelo van en columnas separadas y no pegados en una sola.
VIAJE	Bien después de sacar las paradas. Eran el único campo que iba a quedar con una lista adentro. ruta_general es un texto descriptivo, no una lista.
PUNTO	Bien. direccion y referencia van separadas.
LOCALIDAD	Bien.
PARADA_VIAJE	Bien, cada fila es una parada de un viaje.
SOLICITUD	Bien, cada fila es una solicitud de un pasajero a un viaje.

4.3 Segunda forma normal

Para estar en 2FN hay que estar en 1FN y que ningún atributo dependa solo de una parte de la llave. Eso solo puede pasar si la llave es compuesta.

Nueve de nuestras tablas tienen llave simple (UNIVERSIDAD, PROGRAMA, ESTUDIANTE, CONDUCTOR, PASAJERO, VEHICULO, VIAJE, PUNTO y LOCALIDAD), así que cumplen 2FN automáticamente: no hay partes de la llave de las cuales pudiera depender algo.

Las dos con llave compuesta sí hay que revisarlas:

Tabla	Revisión
PARADA_VIAJE	La llave es (id_viaje, id_punto). El orden no depende solo del viaje, porque un viaje tiene varias paradas con órdenes distintos, ni solo del punto, porque el mismo punto puede ir en distinta posición según el viaje. Con origen_parada pasa igual: un punto puede ser parada INICIAL en un viaje y venir de una SOLICITUD en otro. No hay dependencias parciales.
SOLICITUD	La llave es (id_viaje, documento_pasajero). Ni el punto de recogida, ni la fecha, ni el estado dependen solo del viaje o solo del pasajero: describen esa solicitud en particular. Un pasajero puede pedir que lo recojan en sitios distintos según el viaje, y un viaje puede tener solicitudes en estados distintos según el pasajero. No hay dependencias parciales.

4.4 Tercera forma normal

Para estar en 3FN hay que estar en 2FN y que ningún atributo que no sea llave determine a otro que tampoco lo sea. Eso es lo que se llama dependencia transitiva.

Nosotros evitamos dos de esas desde el diseño, lo que explicamos en la sección 4.6. Con eso, la revisión queda así:

Tabla	Revisión
UNIVERSIDAD	Bien, solo tiene un atributo que no es llave.
PROGRAMA	Bien. El nombre del programa y la universidad son independientes entre sí.
ESTUDIANTE	Bien. Aquí estaba la transitiva documento → programa → universidad, y se quitó al sacar PROGRAMA y UNIVERSIDAD a sus propias tablas. El correo determina lo demás, pero el correo es llave candidata, así que no cuenta como transitiva.
CONDUCTOR	Bien, un solo atributo.
PASAJERO	Bien, un solo atributo.
VEHICULO	Bien, con el supuesto que explicamos en la sección 4.7 sobre marca y modelo.
VIAJE	Bien. Aquí había otra transitiva: id_viaje → placa → conductor. La quitamos no guardando el conductor en VIAJE, porque se puede llegar a él por el carro.
PUNTO	Bien. La dirección no determina la localidad: puede haber calles con el mismo nombre en localidades distintas.
LOCALIDAD	Bien. El nombre no determina la ciudad, porque se puede repetir entre municipios. Por eso la llave alterna es la pareja completa.
PARADA_VIAJE	Bien. orden y origen_parada son independientes.
SOLICITUD	Bien. Aquí decidimos no guardar la tarifa, porque depende del viaje y no de la solicitud: el enunciado dice que el precio es el mismo para todos.

4.5 Forma normal de Boyce-Codd

Una tabla está en FNBC si cada vez que algo determina a otra cosa, ese algo es una superllave. Revisamos las once:

Tabla	Determinantes	Resultado
UNIVERSIDAD	id_universidad, nombre	En FNBC. Los dos son llave.
PROGRAMA	id_programa, (nombre, id_universidad)	En FNBC. Los dos son llave.
ESTUDIANTE	documento, correo_institucional	En FNBC. Tener dos llaves candidatas no es problema porque no se traslapan.
CONDUCTOR	documento	En FNBC.
PASAJERO	documento	En FNBC.
VEHICULO	placa	En FNBC con el supuesto de la sección 4.7. Si aceptáramos que modelo determina marca, ahí sí se saldría.
VIAJE	id_viaje	En FNBC.
PUNTO	id_punto	En FNBC.
LOCALIDAD	id_localidad, (nombre, ciudad)	En FNBC. Los dos son llave.
PARADA_VIAJE	(id_viaje, id_punto), (id_viaje, orden)	En FNBC. Las dos llaves candidatas comparten id_viaje, pero como ambas son llaves completas no hay problema. Solo habría violación si un determinante fuera un pedazo de una llave.
SOLICITUD	(id_viaje, documento_pasajero)	En FNBC.

O sea que las once tablas quedan en FNBC, teniendo en cuenta el supuesto de VEHICULO que explicamos ahora.

4.6 Qué cambiamos para que quedara normalizado

#	Qué hicimos	Qué evitamos
1	Sacamos UNIVERSIDAD y PROGRAMA a sus propias tablas en vez de dejarlas como texto en ESTUDIANTE.	La transitiva documento → programa → universidad (3FN).
2	Sacamos las paradas a PARADA_VIAJE.	Un atributo multivalor dentro de VIAJE (1FN).
3	No guardamos el conductor en VIAJE; se llega a él por el carro.	La transitiva id_viaje → placa → conductor (3FN).
4	Sacamos LOCALIDAD a su propia tabla.	Repetir el nombre de la zona en cada punto (3FN).
5	Pusimos la tarifa en VIAJE y no en SOLICITUD.	Repetir el mismo precio en cada solicitud del mismo viaje.
6	No guardamos los cupos disponibles, se calculan.	Un dato repetido que se puede sacar solo.
7	Juntamos solicitud y reserva en una sola tabla con estado.	Duplicar la relación entre pasajero y viaje.

4.7 El caso de marca y modelo

Hay una cosa en VEHICULO que vale la pena mirar. En la vida real, el modelo determina la marca: un Sandero siempre va a ser Renault. Si aceptamos esa dependencia, tendríamos placa → modelo → marca, que es transitiva, y como modelo no es llave, VEHICULO se saldría de 3FN y de FNBC.

Teníamos dos caminos:

- Separar: crear tablas MARCA y MODELO, y dejar en VEHICULO solo la FK al modelo. Queda perfecto, pero obliga a mantener un catálogo de marcas y modelos que el enunciado nunca pide.
- Dejarlo como está y explicar por qué: RF2 dice que el conductor escribe la marca y el modelo cuando registra el carro, sin escoger de ninguna lista. Como el sistema no controla esa correspondencia, dentro de nuestra base de datos esa dependencia no se cumple, y VEHICULO se queda en FNBC.

Escogimos el segundo. Ningún requerimiento de consulta ni de tablero agrupa por marca, así que meter dos tablas más complicaría el modelo sin servir para nada. Igual lo dejamos escrito para que quede claro que lo vimos y no se nos pasó.

4.8 Sobre 3FN y FNBC

Vale la pena anotar la diferencia: cuando uno descompone hasta 3FN siempre puede conservar todas las dependencias funcionales, mientras que al llegar hasta FNBC a veces toca sacrificar alguna y controlarla desde la aplicación.

En nuestro caso no nos tocó escoger, porque las once tablas llegaron a FNBC sin tener que partir nada: en todas, lo que determina algo resultó ser llave. La única dependencia que nos habría puesto en ese dilema es la de marca y modelo, que ya explicamos.

4.9 Esquema final

Las llaves primarias van primero y las foráneas se marcan con una flecha a la tabla que referencian.

UNIVERSIDAD (id_universidad, nombre)

PROGRAMA (id_programa, nombre, id_universidad → UNIVERSIDAD)

ESTUDIANTE (documento, nombre, apellidos, correo_institucional, celular, id_programa → PROGRAMA)

CONDUCTOR (documento → ESTUDIANTE, fecha_habilitacion)

PASAJERO (documento → ESTUDIANTE, fecha_registro)

VEHICULO (placa, marca, modelo, anio, color, cupos, documento_conductor → CONDUCTOR)

VIAJE (id_viaje, fecha_hora_salida, ruta_general, tarifa, cupos_ofrecidos, estado, distancia_km, hora_salida_real, hora_llegada_real, placa → VEHICULO, id_punto_origen → PUNTO)

PUNTO (id_punto, direccion, referencia, id_localidad → LOCALIDAD)

LOCALIDAD (id_localidad, nombre, ciudad)

PARADA_VIAJE (id_viaje → VIAJE, id_punto → PUNTO, orden, origen_parada)

SOLICITUD (id_viaje → VIAJE, documento_pasajero → PASAJERO, id_punto → PUNTO, fecha_solicitud, estado, fecha_respuesta)

5. Pruebas

Armamos casos de éxito y de falla para revisar que el modelo aguante. Cada uno dice qué restricción o qué regla está probando.

5.1 Llave primaria y datos repetidos

ID	Qué se hace	Qué debe pasar	Prueba
PK-1	Registrar un conductor con un documento que no existe.	Éxito.	PK de ESTUDIANTE
PK-2	Registrar otro conductor con el mismo documento.	Falla, la llave primaria no se puede repetir.	PK de ESTUDIANTE
PK-3	Registrar un carro con una placa nueva.	Éxito.	PK de VEHICULO
PK-4	Registrar otro carro con la misma placa, aunque sea de otro dueño.	Falla.	PK de VEHICULO
PK-5	Meter dos veces la misma parada (mismo viaje, mismo punto).	Falla.	PK compuesta de PARADA_VIAJE
PK-6	Un pasajero pide cupo dos veces en el mismo viaje.	Falla.	PK de SOLICITUD, RN9
ND-1	Registrar un estudiante con un correo que ya usa otro.	Falla.	ND del correo, RN11
ND-2	Poner dos paradas con el mismo orden en un mismo viaje.	Falla.	ND de (id_viaje, orden)

5.2 Llaves foráneas

ID	Qué se hace	Qué debe pasar	Prueba
FK-1	Registrar un carro para un conductor que sí existe.	Éxito.	FK a CONDUCTOR
FK-2	Registrar un carro para un conductor que no existe.	Falla, no puede apuntar a alguien que no está.	FK a CONDUCTOR
FK-3	Publicar un viaje con un carro y un punto que existen.	Éxito.	FK a VEHICULO y PUNTO
FK-4	Publicar un viaje con un punto de origen que no existe.	Falla.	FK a PUNTO
FK-5	Meter en CONDUCTOR un documento que no está en ESTUDIANTE.	Falla, no hay conductor sin estudiante.	FK de la herencia
DEL-1	Borrar un conductor que tiene carros inscritos.	Falla, hay filas que lo referencian.	FK a CONDUCTOR
DEL-2	Borrar un viaje que tiene solicitudes y paradas.	Falla.	FK a VIAJE
DEL-3	Borrar un punto que es origen de un viaje.	Falla.	FK a PUNTO
DEL-4	Borrar un carro que nunca se usó en ningún viaje.	Éxito, nadie lo referencia.	FK a VEHICULO

5.3 Restricciones de chequeo

ID	Qué se hace	Qué debe pasar	Prueba
CK-1	Crear un viaje con estado PROGRAMADO.	Éxito, está en la lista permitida.	CK del estado
CK-2	Crear un viaje con estado 'EN CAMINO'.	Falla, ese valor no existe.	CK del estado
CK-3	Registrar un carro con cupos negativos.	Falla.	CK de cupos
CK-4	Registrar un carro con cupos en 0.	Falla, no podría ofrecer viajes.	CK de cupos
CK-5	Registrar un carro con año 2045.	Falla.	CK del año
CK-6	Crear un viaje con tarifa negativa.	Falla.	CK de tarifa
CK-7	Crear una parada con origen 'MANUAL'.	Falla, solo van INICIAL y SOLICITUD.	CK de origen_parada
CK-8	Crear un viaje sin distancia ni horas reales.	Éxito, esos campos van vacíos hasta que el viaje termine.	Nulos permitidos
CK-9	Terminar un viaje con hora de llegada anterior a la de salida.	Falla.	CK de horas

5.4 Casos por requerimiento

RF	Qué se hace	Qué debe pasar	Prueba
RF1	Registrar un estudiante nuevo como conductor con su carro.	Éxito.	PK, FK
RF1	Registrar un conductor con un correo ya usado.	Falla.	RN11
RF1	Volverse conductor sin inscribir ningún carro.	Falla, la app lo bloquea.	RN10
RF2	Un conductor registra un carro nuevo con 4 cupos.	Éxito.	PK
RF3	Publicar un viaje con 3 cupos en un carro de 4, con dos paradas.	Éxito, las paradas quedan como INICIAL.	RN1
RF3	Publicar un viaje con 5 cupos en un carro de 4.	Falla.	RN1
RF3	Publicar un viaje sin ninguna parada.	Falla, todo viaje debe tener al menos una.	Obligatoriedad
RF4	El conductor acepta una solicitud y hay cupos libres.	Éxito, pasa a ACEPTADA y el punto de recogida se vuelve parada.	RN8
RF4	Aceptar una solicitud cuando ya se llenaron los cupos.	Falla.	RN2
RF4	El conductor rechaza una solicitud.	Éxito, pasa a RECHAZADA y no se crea parada.	RN8
RF5	Pasar un viaje de PROGRAMADO a EN_CURSO y luego a COMPLETADO.	Éxito.	RN5
RF5	Cambiarle el estado a un viaje ya COMPLETADO.	Falla, es estado final.	RN5
RF5	Cancelar un viaje faltando 30 minutos.	Falla, se necesitan 2 horas.	RN4
RF5	Cancelar un viaje con 3 horas de anticipación y 2 pasajeros confirmados.	Éxito, las dos solicitudes quedan canceladas solas.	RN4, RN6

RF	Qué se hace	Qué debe pasar	Prueba
RF6	El conductor consulta sus viajes con los pasajeros confirmados.	Éxito.	Consulta
RF6	Un conductor consulta el historial de otro conductor.	Falla, solo ve los suyos.	RoI
RF7	Un estudiante nuevo crea cuenta de pasajero.	Éxito.	PK, FK
RF8	Un pasajero cambia su celular y su correo a uno libre.	Éxito.	RN11
RF8	Un pasajero intenta cambiarse el documento.	Falla, la llave primaria no se cambia.	PK
RF9	Alguien sin cuenta busca viajes por fecha y zona.	Éxito, ve los viajes PROGRAMADO.	Consulta pública
RF9	Buscar en una fecha sin viajes.	Éxito con resultado vacío, sin error.	Caso borde
RF10	Un pasajero pide cupo en un viaje PROGRAMADO.	Éxito, queda PENDIENTE.	RN3
RF10	Pedir cupo en un viaje que ya arrancó.	Falla.	RN3
RF10	El conductor pide cupo en su propio viaje.	Falla.	RN7
RF10	Alguien sin cuenta intenta pedir cupo.	Falla, toca estar autenticado.	RoI
RF11	Un pasajero cancela su cupo con 3 horas de anticipación.	Éxito, se libera el cupo.	RN4
RF11	Un pasajero cancela faltando una hora.	Falla.	RN4
RF11	Un pasajero intenta cancelar la solicitud de otro.	Falla.	RoI

5.5 Roles

ID	Qué se hace	Qué debe pasar
ROL-1	Un pasajero intenta aceptar una solicitud.	Falla, eso solo lo hace el conductor del viaje (RF4).
ROL-2	Un conductor gestiona solicitudes de un viaje que no es suyo.	Falla.
ROL-3	Alguien sin cuenta consulta los viajes disponibles.	Éxito, RF9 lo permite sin autenticarse.
ROL-4	Alguien sin cuenta intenta ver un tablero de control.	Falla, los tableros son del usuario autenticado.
ROL-5	Un pasajero intenta publicar un viaje.	Falla, RF3 es solo del conductor.
ROL-6	Un estudiante que es conductor y pasajero entra a sus dos tableros.	Éxito, el modelo lo permite porque la herencia tiene traslape.

5.6 Consultas y tableros

Req.	Qué se hace	Qué debe pasar
RFC1	Contar solicitudes por localidad en un rango de fechas.	Éxito, los totales deben cuadrar con un conteo hecho a mano sobre los datos de prueba.
RFC1	Consultar un rango sin solicitudes.	Éxito con resultado vacío, sin error.

Req.	Qué se hace	Qué debe pasar
RFC2	Sumar tarifas de solicitudes aceptadas por localidad y fechas.	Éxito, no deben contar las pendientes, rechazadas ni canceladas.
RFC2	Revisar que un viaje cancelado no sume ingresos.	Éxito, queda por fuera gracias a RN6.
RFC3	Ver los viajes de un conductor filtrando por zona y fechas.	Éxito.
RFC3	Consultar un conductor que no ha publicado nada.	Éxito con resultado vacío.
RFA1	Tablero de un conductor con tres viajes completados.	Éxito: pasajeros llevados, tiempo total (llegada menos salida), ingresos (tarifa por pasajeros) y distancia sumada.
RFA1	Tablero de un conductor sin viajes completados.	Éxito mostrando ceros, no un error.
RFA1	Revisar que un viaje cancelado no sume al tablero.	Éxito, solo cuentan los COMPLETADO.
RFA2	Tablero de un pasajero con cuatro viajes en tres zonas.	Éxito: viajes tomados, zona más frecuente, gasto total y promedios de distancia y tiempo.
RFA2	Tablero de un pasajero sin viajes.	Éxito mostrando ceros, no un error.
RFA2	Pasajero con empate entre dos zonas.	Éxito, pero hay que tener definido cómo se desempata.

Con esto quedan cubiertos los once RF, los tres RFC, los dos RFA, las once reglas de negocio, las restricciones de las once tablas, inserciones y borrados, y los tres roles de usuario.